CAMP

Consumption-Aware Memory Prediction for Scientific Workflows

Technical Report and Artifact Reproduction Guide

Version 1.0

August 2026

Abstract

Scientific workflow schedulers must request memory before a task executes. Static input size, task identity, and prior peak-memory observations are available before launch, but they do not directly describe how many bytes a program will actually consume. CAMP augments these signals with task-attributed consumed-data measurements collected from successful read-family system-call returns and file-backed memory-mapped page faults. Only completed observations enter the prior history used for later decisions.

This report presents four connected evaluations. RQ1 compares full STRACE and eBPF auditing across six workflows and shows that mmap activity can add substantial signal beyond explicit reads, although full eBPF auditing can impose material runtime overhead. RQ2 compares two memory-allocation configurations on 5,026 held-out tasks. The Access Baseline uses static prelaunch information and prior peak-RSS history (A+P). CAMP extends the Access Baseline with prior consumed-data history and a prelaunch estimate of current consumption (A+P+C). The remainder of this report refers to these configurations as Access Baseline and CAMP. CAMP reduces first-attempt underallocations from 87 with the Access Baseline to 31, a 64.37% reduction, while both configurations resolve all tasks after retry. RQ3 compares CAMP with the unchanged Sizey implementation on the same 21,337 test tasks and assignments. CAMP reduces first-attempt underallocations from 3,597 to 287 and median absolute percentage error from 27.04% to 16.14%, at the cost of higher requested memory. RQ4 packages a reproducible equal-budget evaluation of a three-gate selective-auditing policy against uniform and process-stratified random selection.

Keywords: scientific workflows; memory prediction; resource allocation; eBPF; STRACE; selective auditing; Nextflow; Slurm; reproducibility

Contents

1. Executive Summary

CAMP treats workflow memory allocation as both a prediction problem and a measurement problem. A scheduler knows which files are staged before launch, but a selective parser may touch only part of a file and a memory-mapped program may access pages without conventional read calls. CAMP measures successful explicit reads and attributable file-backed mmap page faults, stores only completed outcomes, and makes those observations available to later comparable tasks.

- Audit visibility: eBPF captures mmap-backed consumption absent from STRACE explicit-read accounting. The contribution is especially important for Minimap2, GATK, and Viralmetagenome.
- Audit cost: full eBPF overhead ranges from approximately 5.54% for GATK to 138.85% for Minimap2 in the packaged comparison, motivating selective measurement.
- RQ2 allocation safety: CAMP reduces first-attempt underallocations from 87 with the Access Baseline to 31 on 5,026 held-out tasks; both configurations have zero unresolved tasks after retry.
- RQ3 external comparison: CAMP records 287 first-attempt underallocations compared with Sizey's 3,597 and reduces median absolute percentage error from 27.04% to 16.14%.
- RQ4 selective-auditing package: fixed risk, information, and discovery gates are evaluated at six budgets with 1,000 equal-budget repetitions (1000 repetitions with randomly selected tasks change for 3 policies, i.e. CAMP, Uniform Random Sampling and Process stratified Random Sampling for every repetition.)

2. Introduction

2.1 Motivation

Scientific workflows decompose analyses into scheduler tasks with heterogeneous memory behavior. A resource manager such as Slurm requires a memory request before launch. A request below peak resident set size can cause failure or retry; an unnecessarily large request reserves capacity that cannot serve other work. Submission-time estimators use workflow identity, declared resources, input sizes, and prior peak memory, but these features cannot fully represent selective reading, decompression, streaming, caching, or mmap access.

2.2 Problem Statement

For task t, the Access Baseline uses prelaunch static information and peak-RSS summaries from completed comparable tasks. CAMP extends it with completed consumed-data history and a prelaunch estimate of current consumption. CAMP asks whether this additional information can produce safer calibrated memory requests without using the current task's outcome. Current peak RSS and measured consumed bytes are labels revealed only after the task completes and cannot determine their own allocation.

2.3 Scope and Research Questions

- RQ1 - Audit signal and cost: How do STRACE and eBPF differ in consumed-data visibility and runtime overhead under full auditing?
- RQ2 - Consumption-aware allocation: How does CAMP change held-out allocation safety and capacity cost relative to the Access Baseline?
- RQ3 - External comparison: How does CAMP compare with Sizey on identical tasks and assignments?
- RQ4 - Selective auditing: How should a three-gate selector be evaluated against equal-count uniform and process-stratified random policies for the Access Baseline and CAMP?

2.4 Contributions

1. A task-attributed consumed-data signal combining successful explicit-read returns with file-backed mmap page-fault bytes.
2. A held-out allocation experiment using prior information to isolate the effect of consumed-data feedback.
3. A matched comparison with the Sizey implementation.

4. A selective-auditing experiment with fixed budgets, policies, repetitions, inputs datasets, and output metrics.

3. CAMP Design

3.1 Information Timing

We get the information below according to different timings during a workflow run.

A is obtained before the launch of a workflow, while P and C are histories available before a current task runs.

While the outcomes are obtained once a task is complete.

Symbol	Information	Availability
A	Workflow/process identity, version, system configuration, staged bytes, requested resources, and domain fields	Before launch
P	Counts, confidence, quantiles, and similarity summaries from completed peak-RSS observations	Before current decision
C	Consumed-data histories from completed audited tasks plus predicted current consumed bytes	Before current decision
Outcomes	Current peak RSS, runtime, explicit reads, mmap-fault bytes, and retry state	After completion only

3.2 Consumed-data Measurement

For a completed task, CAMP defines consumed data as the sum of positive read-family syscall return bytes and attributable file-backed mmap page-fault bytes. Failed calls and requested byte counts are excluded. STRACE supplies a user-space syscall record useful for comparison, while eBPF provides kernel-level task-attributed events and observes mmap-backed access absent from explicit reads.

```
consumed_bytes = successful_read_return_bytes + file_backed_mmap_page_fault_bytes
```

3.3 Prior History

Completed observations are summarized from specific task-instance through broader process, workflow, and system-configuration scopes. A task uses the most specific supported history and falls back when evidence is sparse. The SQLite seed contains 28,490 completed development outcomes. The DB contains only 28k training data initially, and a test task outcome is added after it completes, allowing later tasks to use it as a prior history.

3.4 Memory Prediction and Allocation

Memory prediction estimates likely peak RSS, while allocation converts those estimates into a safety-oriented scheduler request. Treating these as separate stages is necessary because the request that minimizes central prediction error is not necessarily the request that controls upper-tail failure risk.

3.4.1 Prediction inputs

The Access Baseline uses categorical workflow, process, version, and system identifiers together with numeric task and staged-input fields and prior peak-RSS summaries. CAMP adds prior consumed-data summaries, consumption-to-input history, and a prelaunch estimate of current consumed bytes. Both configurations explicitly exclude the current task's peak RSS, measured consumption, runtime, read/fault bytes, and observed OOM state.

3.4.2 Model training and out-of-fold prediction

LightGBM quantile regression is used inside preprocessing pipelines. Numeric fields receive median imputation with missing indicators; categorical fields receive most-frequent imputation and one-hot encoding with unknown-

category support. Frozen settings use five grouped folds, three ensemble seeds, 350 estimators, learning rate 0.03, and quantiles 0.50, 0.90, 0.95, 0.99, and 0.995.

Grouped out-of-fold prediction means that each development row is predicted by a model that did not train on that row's group. These leakage-controlled predictions support calibration. Final models then fit the development population and predict held-out tasks. CAMP's current-consumption estimate is also predicted from static and prior information; measured current consumption remains an outcome.

3.4.3 Allocation calibration

Calibration evaluates candidate bases from the median point estimate, historical-neighbor p95/p99 values, and model q90/q95/q99/q99.5 predictions. Each candidate may receive a multiplicative correction from historical log residuals at fixed quantiles between 0.90 and 0.999, and the result is rounded upward to MiB. Development rows are deterministically divided by hashed task identity: 60% build residual references and 40% compare allocation candidates.

For every task in the 60% reference subset, CAMP compares the observed peak memory with each candidate base and records the difference as a log ratio: log(observed peak divided by candidate base). A positive residual means that the candidate was too small, while a negative residual means that it exceeded the observed peak. CAMP summarizes the relevant historical residuals at each configured quantile and converts the log value back into a multiplicative correction factor. Negative corrections are clipped to zero, so calibration may increase a candidate base but never reduce it.

For every task in the 40% selection subset, CAMP multiplies each candidate base by its corresponding residual correction factor and rounds the result upward to the next whole MiB. It then checks whether this first request covers the task's observed peak, calculates coverage across all selection tasks and separately within each workflow, and calculates total requested memory relative to total observed peak memory. These results are used only to compare allocation candidates; held-out test tasks remain unused.

A candidate allocation policy is eligible when it achieves at least 99% first-request coverage across all development-selection tasks and at least 97.5% first-request coverage within every workflow. Among eligible candidates, the procedure chooses the smallest request-to-observed-peak ratio, followed by fewer underallocations, a lower residual quantile, and stable base-policy name ordering. If no candidate meets both coverage conditions, the procedure selects the best available candidate and records that the constraints were unmet. Held-out test outcomes do not participate in policy selection.

3.4.4 Underallocation, coverage, and retry

A task is underallocated when its first memory request is below recorded peak RSS. First-attempt coverage is one minus the underallocation fraction. For an insufficient first request, the common retry procedure raises the request to at least 1.5 times its prior value; available exact or neighboring historical p99 values may raise it further. The process stops when the request covers recorded peak RSS or reaches a 20-retry limit. The same fixed rule is applied to every paired method.

3.5 Selective Auditing

- Risk lane (80%): likely underallocations and large expected memory shortfalls.
- Information lane (10%): tasks expected to expand useful consumed-data knowledge.
- Discovery lane (10%): exploration of sparse or distant task regions.

4. Experimental Method

4.1 Workflow Cohort

Workflow	Matched logical tasks

GATK	500
Minimap2	6,001
Sarek	6,012
Seqinspector	6,001
Taxprofiler	5,999
Viralmetagenome	9,003
Total	33,516

Custom workflow definitions are packaged for GATK and Minimap2. Rest of the workflows are taken from nextflow/nfcore github repositories.

4.2 Experimental Platform

Experiment	Recorded or required configuration
RQ1	One coordinator plus 13 Slurm workers; 32 vCPUs and 256 GiB RAM per worker, Nextflow, Slurm, Apptainer; STRACE for Exp1; eBPF for Exp2.
RQ2	Linux x86-64; Python 3.9+; completed repeated jobs used 32 CPUs and 220 GiB RAM; packaged single-seed command uses 8 CPU threads.
RQ3	Linux x86-64, Python 3.9; Sizey used 32 CPUs/220 GiB per workflow; final CAMP used 32 CPUs/200 GiB, three-hour limit, and 32 CPU threads.
RQ4	Single Linux x86-64 host; archived Python 3.9.25 environment; 8 CPU threads; no Slurm or live workflows;

4.3 Data Partitions and Leakage Controls

RQ2 assigns 28,490 of 33,516 chronological tasks to development and 5,026 to held-out testing. Grouped out-of-fold prediction prevents a row's model from training on that row. RQ3 and RQ4 use a 30,478-task Sizey-compatible cohort with 9,141 initial and 21,337 later tasks under seed 1996 (3k records were reduced because (workflow process): (Viralmetagenome: Kreport2Krona) had task instances runtime were). Paired methods use identical task identifiers and assignments. Model and policy choices are frozen before test evaluation.

4.4 Metrics

- Consumed-data visibility: explicit-read GiB, mmap-fault GiB, total GiB, and mmap percentage.
- Runtime overhead: wall-time increase relative to matching unaudited execution.
- Safety: underallocations, first-attempt coverage, retries, and unresolved tasks.
- Capacity: total requested memory, request-to-peak ratio, and runtime-weighted unused memory-time.
- Prediction: mean absolute error, median absolute percentage error, and log-scale R^2.
- Audit yield: recovery of risky, informative, heavy, or workflow-specific tasks at equal task counts.

RQ2 and RQ3 underallocations are offline replay against recorded peak RSS.

5. RQ1: Audit Signal and Runtime Cost

5.1 Objective and Protocol

RQ1 compares three matching modes: Exp0 without runtime audit, Exp1 with full STRACE auditing, and Exp2 with full eBPF auditing. Modes use matching inputs, process resources, worker allocations, and software. Overhead measurements uses the runtime of workflows of Exp1 and Exp2 relative to Exp0. STRACE summaries retain successful read-family returns; eBPF summaries combine read-return and mmap-fault bytes.

5.2 Consumed-data Visibility

Across the six workflows, eBPF records 511.89 GiB of mmap-fault activity absent from STRACE explicit-read accounting. The mmap share ranges from about 6.61% for Taxprofiler to 50.69% for Minimap2. Minimap2 illustrates the difference: STRACE records 51.59 GiB of explicit reads; eBPF records 50.93 GiB of explicit reads plus 52.37 GiB of mmap faults, producing 103.30 GiB total, 100.23% above the STRACE explicit-read measurement.

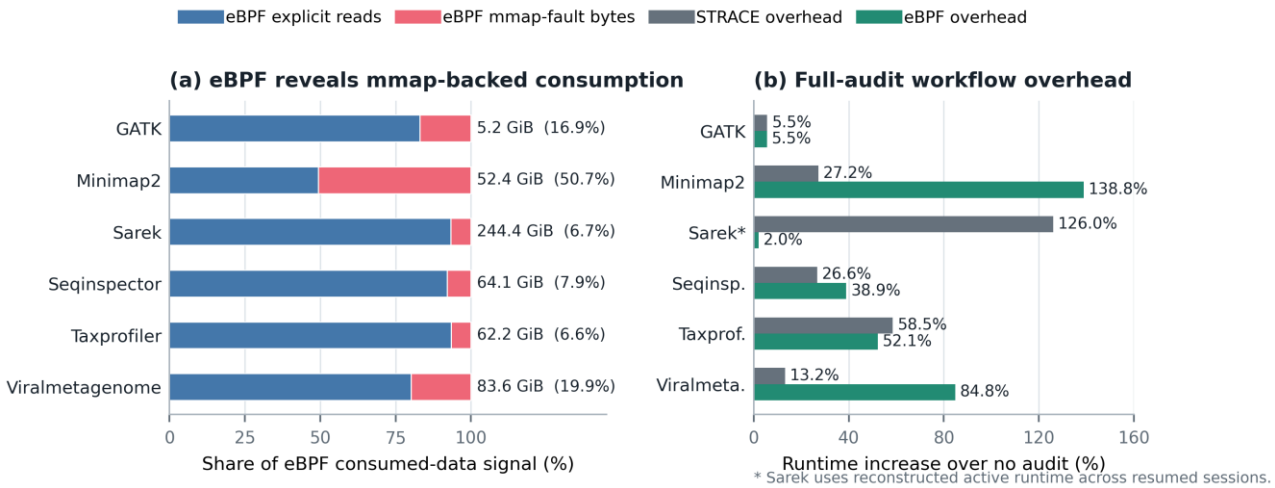

Figure 1. RQ1 consumed-data visibility and full-auditing overhead across packaged workflow runs.

5.3 Runtime Overhead

Workflow	STRACE	eBPF
GATK	5.55%	5.54%
Minimap2	27.18%	138.85%
Sarek	126.05%	1.99%
Seqinspector	26.64%	38.86%
Taxprofiler	58.55%	52.15%
Viralmetagenome	13.25%	84.82%

5.4 Interpretation

The primary eBPF advantage is visibility, not uniformly lower cost. It captures mmap-backed consumption missed by explicit-read tracing but overhead varies substantially by workload. These observed costs motivate selective auditing.

6. RQ2: Effect of Consumption-aware Feedback

6.1 Objective and Compared Views

- Access Baseline: static prelaunch information plus prior peak-RSS history.
- CAMP: Access Baseline plus prior consumed-data history and a prelaunch estimate of current consumption.

Both views use the same development rows, same 5,026 held-out rows, same model family, and same decision ordering. Current outcomes remain hidden until after each allocation. This is a offline replay using the prior information of peak RSS which were run on workflows.

6.2 Held-out Safety

Metric	Access Baseline	CAMP	Change
Test tasks	5,026	5,026	Same cohort
First-attempt underallocations	87	31	−64.37%
First-attempt coverage	98.27%	99.38%	+1.11 points

CAMP substantially reduces first requests below recorded peak RSS. The result supports consumed-data history as an upper-tail safety signal for this cohort.

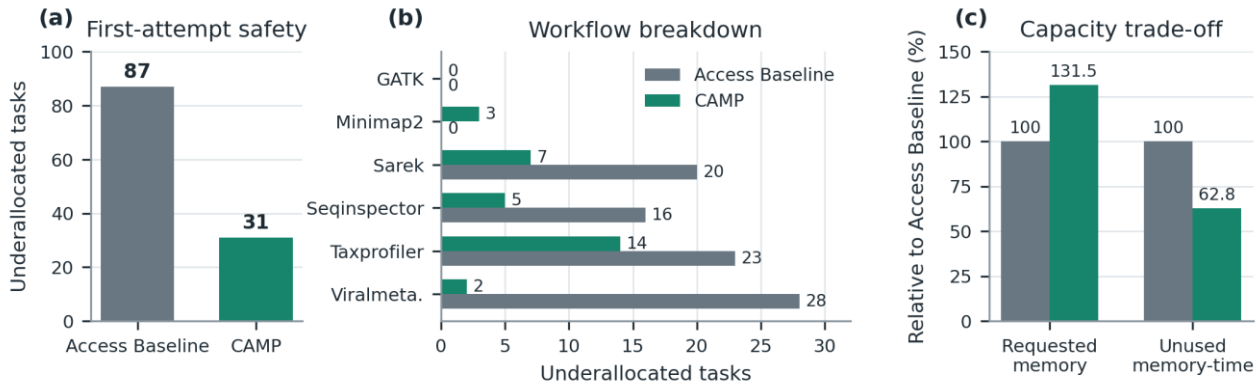

Figure 2. RQ2 safety and capacity tradeoff for the Access Baseline and CAMP on 5,026 held-out tasks.

6.3 Capacity and Prediction Tradeoffs

Relative to the Access Baseline, CAMP requests 31.50% more total memory while runtime-weighted unused memory-time ((Requested Peak RSS – Original Peak Rss) * Runtime of task) decreases by 37.18%. The additional reservation is concentrated on tasks judged to be risky instead of being applied uniformly. Point-estimation results show a scale-dependent trade-off. CAMP reduces mean absolute error from 1,089.95 MiB to 998.31 MiB—a reduction of 91.63 MiB or 8.41%. The largest reductions occur for Taxprofiler, where MAE decreases by 327.77 MiB, and Seqinspector, where it decreases by 222.43 MiB. These relatively large absolute-error reductions strongly influence the overall mean. However, median absolute percentage error increases from 21.75% to 26.51%, a deterioration of 4.76 percentage points. This increase is concentrated in the numerous small-memory Viralmetagenome tasks, whose median percentage error rises from 6.27% to 20.95%, and Minimap2 tasks, where it rises from 38.89% to 43.37%. For tasks using only a few MiB, even a small prediction difference produces a large percentage error. Thus, CAMP improves average error measured in MiB while worsening the typical proportional error. These are point-prediction results; CAMP's safer allocations come from calibrated upper-tail estimates and residual bounds, which reduce first-attempt underallocations from 87 to 31 despite the higher median percentage error. The principal RQ2 claim is safer calibrated allocation, not dominance in every central-prediction metric.

6.4 Robustness Across Seeds and Splits

The artifact includes five seeds over 30/70, 50/50, 70/30, 85/15, and 90/10 chronological development/test partitions. The non-monotonic pattern occurs because increasing the development fraction also moves the chronological test boundary, changing both the available history and the future task population; it is not a simple learning curve on one fixed test set. At 30/70, history is limited; 50/50 provides the best balance; 70/30 repeatedly selects a stable but insufficiently conservative policy; 85/15 selects different policies across seeds, causing high variation; and 90/10 uses a stable policy that fits its later holdout poorly. Because underallocation is a threshold event, small changes in predictions or calibration can move many tasks from covered to underallocated. The figure therefore demonstrates why CAMP requires repeated-seed evaluation and stable policy calibration rather than showing that additional training data is inherently harmful.

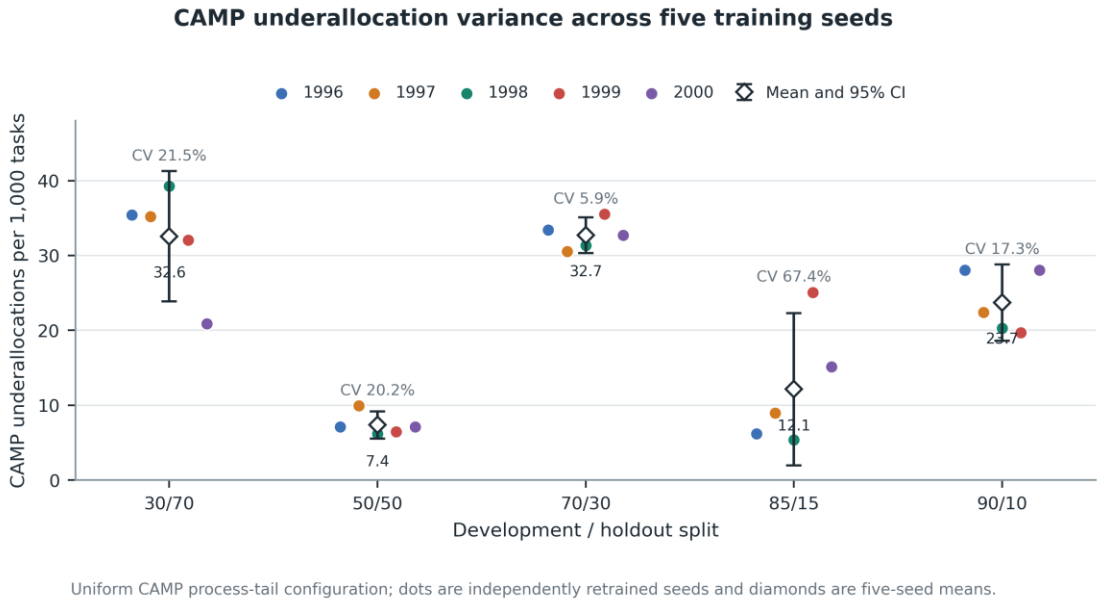

Figure 3. CAMP-only seed variation in first-attempt underallocations across chronological splits.

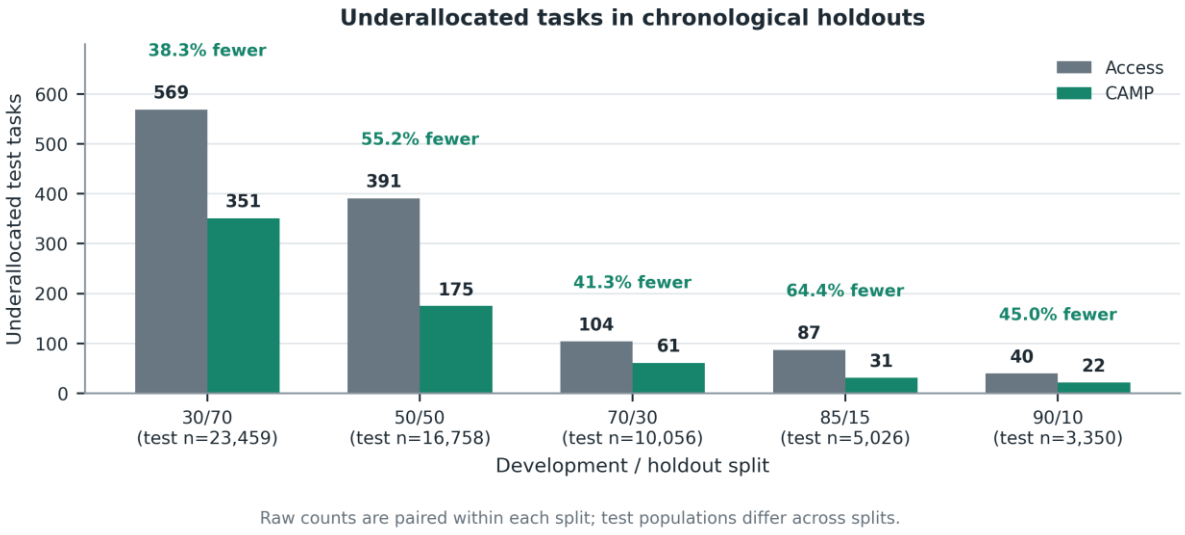

Figure 4. Comparison of ACCESS Baseline and CAMP first-attempt under allocations across chronological splits.

7. RQ3: CAMP versus Sizey

7.1 Objective and Fairness Controls

RQ3 uses the Sizey implementation pinned to commit e0dd09f09cd2bf5905c7143792e3500c948eb386. The primary cohort has 30,478 eligible tasks; seed 1996 assigns 9,141 to initial training and 21,337 to testing. Each method receives identical targets, workflow-specific assignments, and task identities. Outcomes are revealed only after each test decision.

7.2 Execution Protocol

Sizey is executed and normalized separately for the six workflows. CAMP then fits its point and process-tail models and summarizes both methods into one task-matched comparison. The final CAMP job uses 32 model workers.

7.3 Safety and Prediction Results

Metric	Sizey	CAMP	Result
Test tasks	21,337	21,337	Identical cohort
First-attempt underallocations	3,597	287	−92.02%
First-attempt coverage	83.14%	98.65%	+15.51 points
Median absolute percentage error	27.04%	16.14%	−40.31%
Log-scale R^2	0.372	0.617	Higher for CAMP
Unresolved after retry	0	0	None

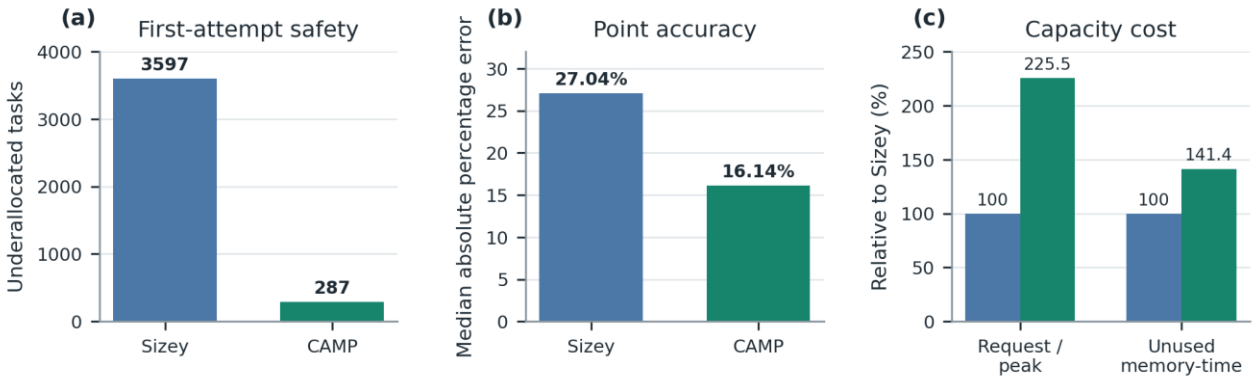

Figure 4. 21,337-tasks: Sizey-versus-CAMP comparison.

7.4 Capacity Tradeoff

CAMP's safety improvement uses more reserved memory. The request-to-observed-peak ratio rises from 1.36× for Sizey to 3.06× for CAMP. Runtime-weighted unused memory-time rises from 31.93 to 45.16 GiB-hours. The result supports CAMP's tail-safety objective but does not show uniform memory-efficiency dominance over Sizey.

8. RQ4: Selective-auditing Experiment

8.1 Objective

RQ4 tests whether fixed risk, information, and discovery gates find higher-value tasks than random policies at the same exact audit count. It runs separately for the Access Baseline and CAMP because available consumed-data history changes both allocation risk and information value.

8.2 Three-gate Policy

The policy assigns 80% of the budget to risk, 10% to information, and 10% to discovery. Risk scoring combines underallocation probability and expected shortfall ranks. Information targets tasks expected to add useful consumed-data evidence. Discovery preserves exploration of sparse regions. Both allocation policies use hierarchical residual calibration with model_q99 and a 0.95 residual quantile.

8.3 Equal-budget Evaluation

Gate models are cross-fitted on 9,141 initial tasks and fitted to score 21,337 later tasks from six workflows. The CAMP three-gate policy is compared with uniform random and process-stratified random selection at exact budgets of 5%, 10%, 20%, 30%, 50%, and 100%, corresponding to 1,067; 2,134; 4,267; 6,401; 10,668; and 21,337 selected tasks per repetition. Exact-count matching prevents recall gains from being caused by auditing more tasks.

For every budget and policy, the primary experiment performs 1,000 deterministic-seed Monte Carlo selections. The tasks, fitted gate scores, policies, and exact audit count remain fixed; only randomized task choices change. Uniform and process-stratified selections change throughout, while the three-gate selector changes only its exploration portion.

8.5 Retained result and evidence boundary

The retained plotting CSV contains 18,000 repetition rows and supports direct regeneration of the displayed PNG and PDF. At 5%, 10%, 20%, 30%, and 50% budgets, mean CAMP three-gate underallocation-target recall is 7.52%, 12.88%, 27.48%, 39.90%, and 57.85%, respectively. Uniform and process-stratified means remain near their nominal budget fractions. The 100% rows are a convergence check in which every policy selects every task. Larger task-level and diagnostic tables remain regenerated working outputs rather than all being embedded in the plotting CSV.

Budget	Three-gate	Uniform	Process-stratified
5%	7.52%	5.02%	4.98%
10%	12.88%	10.02%	10.05%
20%	27.48%	19.88%	19.87%
30%	39.90%	30.17%	29.97%
50%	57.85%	49.90%	49.92%

Figure 5. Mean underallocation-target recall across 1,000 equal-budget selections; random-policy shading shows empirical intervals

9. Discussion

The experiments show why workflow memory allocation should be treated as a feedback system. Static features and prior peak RSS have broad availability, while consumed-data measurements add behavior that staged bytes cannot describe. The gain is strongest as an upper-tail signal: CAMP can concentrate additional capacity on risky tasks and reduce runtime-weighted unused capacity even when aggregate requested memory rises.

RQ1 also establishes that measurement is itself a resource decision. eBPF improves visibility for mmap-heavy workloads, but continuous full auditing can be expensive. Risk, information, and discovery gates address immediate unsafe allocations, insufficient consumed-data knowledge, and blind spots caused by exploitation-only selection.

The Sizey comparison supplies an external reference point. CAMP improves safety and median percentage error but requests more memory. Sizey prioritizes conserving cluster memory. The preferred system depends on whether task failures or excess reserved memory are more costly.

10. Conclusion

CAMP augments scientific-workflow memory prediction with data-consumption evidence from completed tasks. It combines successful explicit reads with file-backed mmap faults, maintains prior histories, and uses calibrated upper-tail predictions to allocate later tasks. The packaged evidence shows that eBPF exposes consumption missed by explicit-read tracing; CAMP reduces RQ2 first-attempt underallocations from 87 with the Access Baseline to 31; and, relative to Sizey, CAMP reduces RQ3 underallocations from 3,597 to 287 and median absolute percentage error from 27.04% to 16.14%, with higher requested capacity.

The RQ4 package extends the design to selective measurement through fixed risk, information, and discovery gates. Together, this report measured consumption, prior history, safety-oriented allocation, and budgeted future measurement.

References

[1] P. Di Tommaso et al., “Nextflow enables reproducible computational workflows,” Nature Biotechnology, vol. 35, no. 4, pp. 316–319, 2017. doi: 10.1038/nbt.3820.

[2] A. B. Yoo, M. A. Jette, and M. Grondona, “SLURM: Simple Linux Utility for Resource Management,” in Job Scheduling Strategies for Parallel Processing, LNCS 2862, pp. 44–60, 2003. doi: 10.1007/10968987_3.

[3] J. Bader et al., “Sizey: Memory-Efficient Execution of Scientific Workflow Tasks,” in 2024 IEEE International Conference on Cluster Computing, pp. 370–381, 2024. doi: 10.1109/CLUSTER59578.2024.00039.

[4] G. Ke et al., “LightGBM: A Highly Efficient Gradient Boosting Decision Tree,” in Advances in Neural Information Processing Systems, vol. 30, pp. 3146–3154, 2017.